UNIVERSITI POLY-TECH MALAYSIA
UPTM

Name: 1. **SAIDATUL NAZIRAH BINTI RAZAK** 2. **ADRIANA KHAYRIN BINTI MOHD SIDEK** 3. **NUR AINA BATRISYA BINTI ZULFIKAR** 4. **QISTINA BATRISYIA BINTI MOHD NAJIB**		
ID number : 1. **AM2412018077** 2. **AM2412018107** 3. **AM2412017932** 4. **AM2412018123**		
Lecturer: **MADAM NOORNAJWA BINTI MD AMIN**		
Course and Course Code: **SWC3684 – DATA STRUCTURES**	Submission Date: 30th March 2026	
Assignment No./Title: **Construct an application by applying appropriate data structures and algorithms.**	Extension & Late Submission: A video presentation is submitted a day after	
Assignment Type: **GROUPING**	% of Assignment Mark : **30%**	Returning Date:

Penalties:

1. 10% of the original mark will be deducted for every one-week period after the submission date

2. No work will be accepted after two weeks of the deadline

3. If you were unable to submit the coursework on time due to extenuating circumstances, you may be eligible for an extension

4. Extension will not exceed one week

Declaration: I/we the undersigned confirm that I/we have read and agree to abide by these regulations on plagiarism and cheating. I/we confirm that this piece of work is my/our own. I/we consent to appropriate storage of our work for checking to ensure that there is no plagiarism/ academic cheating.

Signature:

Name: (SAIDATUL) (ADRIANA) (TISYA) (QISTINA)

This section may be used for feedback or other information

1. INTRODUCTION 3
2. IPO ANALYSIS 5
3. DESIGN UML CLASS DIAGRAM AND FLOWCHART 6
4. SAMPLE DATA IN .TXT FILE, JAVA CODE AND SAMPLE INPUT/OUTPUT. 7
Input: 30
5. LESSON LEARNED FROM EACH MEMBER 33
6. REFERENCES (APA FORMAT) 35
7. VIDEO PRESENTATION LINK 35

Table of Contents

DATA STRUCTURES (SWC3684/SWC3344)

PROJECT RUBRIC

STUDENT NAME: ADRIANA KHAYRIN BINTI MOHD SIDEK
STUDENT NAME: QISTINA BATRISYIA BINTI MOHD NAJIB
STUDENT NAME: NUR AINA BATRISYA BINTI ZULFIKAR
STUDENT NAME: SAIDATUL NAZIRAH BINTI RAZAK
SECTION: 02 CC101

Sect.	Criteria	Max Score	Marking Criteria & Score					Scores	
			No evidence	Weak	Fair	Good	Very Good		
A.	**Problem Definitions and analysis**	**20**	No evidence of problem definitions and analysis.	It is not in own words and the documentation is simply comments embedded in the code and does not help the reader understand the code.	The problem definitions and analysis given in own words, but is minimal and documentation consists of embedded comment and documentation is somewhat useful in understanding the code.	The problem definitions and analysis are not given in own words and the documentation is simply comments embedded in the code with some simple header comments separating routines.	The problem definitions and analysis are given in own words, but is minimal and documentation consists of embedded comment and documentation is somewhat useful in understanding the code.		
			0	**1 – 5 marks**	**6 – 10 marks**	**11 – 15 marks**	**16 – 20 marks**		
B.	**Design UML or appropriate class diagrams**	**20**	No evidence of design section	Most design (algorithms)are illustrated but irrelevant	Some of the steps in the algorithms are incorrect	Some of the steps in the algorithms are correct but incomplete work	Illustrate overall design (algorithms) clearly, completely and correctly		
			0	**1 – 5 marks**	**6 – 10 marks**	**11 – 15 marks**	**16 – 20 marks**		
C.	**Implements the concepts given in the programs**	**20**	No evidence of the implementation	Show evidence of the implementation but all the items are irrelevant	Some of the items are irrelevant, unclear or incomplete	Show all the items appropriately	Show all the items clearly and fully appropriate for the project.		
			0	**1 – 5 marks**	**6 – 10 marks**	**11 – 15 marks**	**16 – 20 marks**		
D.	**Development (coding evidence)**	**10**	No evidence of development	Very poorly written and organized code, inappropriate use of identifiers.	Poorly written and organized code with minimal comments, but, appropriate use of identifiers.	Adequately written and organized code, appropriate use of identifiers, clear, neat & inadequate comments.	Well written and organized code, appropriate use of identifiers, clear, neat, & adequate and helpful comments.		
			0	**1 – 4 marks**	**5 – 7 marks**	**8 – 9 marks**	**10 marks**		
E.	**Testing (output evidence through presentation and documentation)**	**20**	No evidence testing section	Still contain error or warning. Program is completely incorrect.	Evidence of inadequate and incomplete Output.	Runs but with some glitches. Program follows the original partially or gives some incorrect output	Runs smoothly and without glitches. Program is according to the original question and gives correct output		
			0	**1 – 5 marks**	**6 – 10 marks**	**11 – 15 marks**	**16 – 20 marks**		
F.	**Reflective comments/ Problems encountered/ Lesson learned/ Conclusion**	**10**	Fail to provide a conclusion and overall reflective comment/ problems/ lesson learned.	Does not articulate any of the three – what worked well and why, what did not work well and why, how to improve. Discusses on reflective comments either group experience or self-experience in completing the project only. No conclusion provided.	Minimal articulates what worked well and why, what did not work well and why, and considering self-reflective comments as well as group experience and conclusion.	Adequate explanation of stating what works and why, what doesn't work well and why, and consider self-reflective reviews as well as group experiences and conclusions.	Good explanation of stating what works and why, what doesn't work well and why, and consider self-reflective reviews as well as group experiences and conclusions.		
			0	**1 – 4 marks**	**5 – 7 marks**	**8 – 9 marks**	**10 marks**	**GRAND TOTAL (Criteria A until E + Criteria F)**	30%
	Student 1							0	0
	Student 2							0	0
	Student 3							0	0
	Student 4							0	0

1. INTRODUCTION

In today's digital world, cybersecurity has become increasingly important as organizations rely heavily on technology to store and manage data. With the rapid growth of the internet and digital systems, the risk of cyber threats such as phishing, ransomware, distributed denial-of-service (DDoS) attacks, and data breaches has also increased. These incidents can cause serious financial losses, damage to reputation, and disruption of operations. Therefore, it is important for organizations to have an efficient system to detect, manage, and resolve cybersecurity incidents.

A Security Operations Center (SOC) plays a key role in handling these cybersecurity threats. It is responsible for monitoring systems, analyzing potential risks, and responding to incidents in a timely manner. In a SOC environment, analysts are assigned different incidents based on their expertise and workload. Proper management of these tasks is very important to ensure that incidents are resolved quickly and efficiently without overloading any analyst.

This project focuses on developing a Java-based Cybersecurity Incident Management System that simulates the operations of a SOC. The system is designed to manage analyst

information, assign incidents, and track the resolution process. It reads incident data from an input file and organizes the data in a structured way. Each analyst may handle multiple incidents, and the system ensures that all incidents are properly recorded and assigned.

To achieve this, several data structures are used in the system. A LinkedList is used to store and manage the list of analysts along with their assigned incidents. This allows dynamic data storage and easy insertion of new records. Queue data structures are used to assign analysts into different categories based on the number of incidents they are handling. Analysts with fewer incidents are distributed between internal and external queues, while those with a higher number of incidents are assigned to a critical queue. This helps in balancing the workload and prioritizing more critical cases.

In addition, a Stack is used to track the completion of incidents. Once an analyst has completed all assigned tasks, they are pushed into a stack to record the resolution process. This follows the Last-In-First-Out (LIFO) principle, which helps in organizing and displaying completed work efficiently. The combination of LinkedList, Queue, and Stack demonstrates how different data structures can be applied together to solve real-world problems.

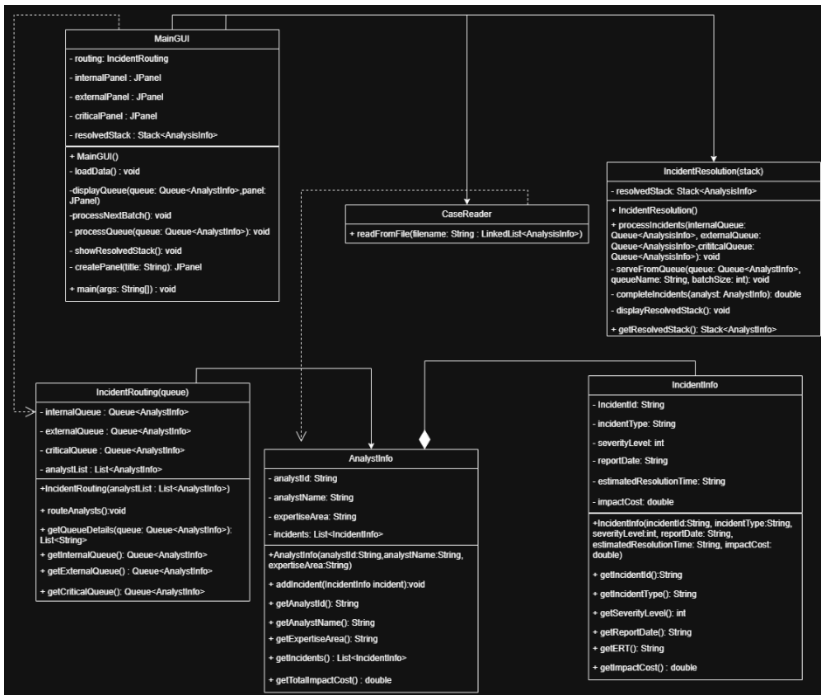
The main objective of this project is to apply the concepts of Abstract Data Types (ADTs) and data structures in developing a functional system. It also aims to improve understanding of how data can be organized, processed, and managed effectively. Through this project, students can gain hands-on experience in system design, problem-solving, and teamwork while building a practical application related to cybersecurity management.

2. IPO ANALYSIS

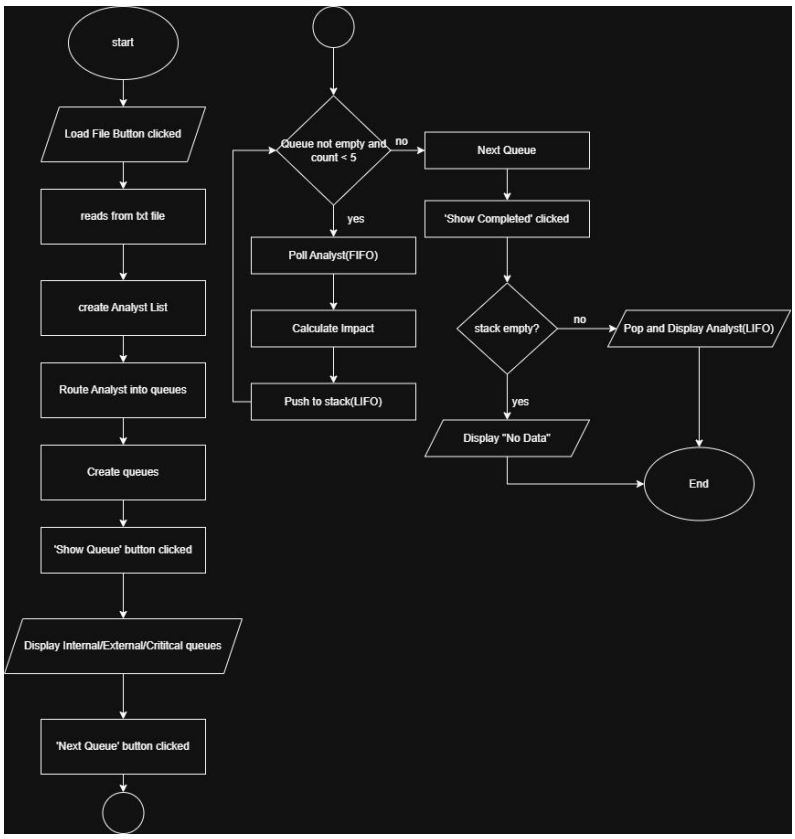
Input	Process	Output
Incident data from text file (cyber_incidents.txt)	Read and parse data from file	Structured analyst and incident data
Analyst and incident information	Store data using LinkedList	Organized list of analysts with incidents
Number of incidents per analyst	Assign analysts into queues (Internal, External, Critical)	Categorized analyst queues
Queue data	Process analysts in batches using FIFO method	Processed analysts
Completed analysts	Store in Stack (LIFO)	Display resolved incidents and total impact cost

3. DESIGN UML CLASS DIAGRAM AND FLOWCHART

UML CLASS DIAGRAM



FLOWCHART



4. SAMPLE DATA IN .TXT FILE, JAVA CODE AND SAMPLE INPUT/OUTPUT.

Sample Data in .txt File

The system uses a text file named cyber_incidents.txt as input. Each line in the file represents one cybersecurity incident along with the analyst's information. The data is separated using vertical bar("|").

Cyber_incidents.txt :

A001|Dean Smith|Network Security|INC001|Port Scanning Attack|2|2009-03-26|3|5000

A002|Castiel Novak|SIEM Monitoring|INC002|Brute Force Attack|3|2010-09-06|7|25000

A003|Sam Smith|Digital Forensics|INC003|Insider Threat|4|2009-03-26|5|100000

A004|Jack Kline|Network Security|INC004|DDoS|4|2017-05-18|5|40000

A005|Claire Smith Novak|Digital Forensics|INC005|Unauthorized Access|3|2023-04-09|5|30000

A006|Crowley Macleod|Malware Analysis|INC006|Spyware|3|2020-05-30|6|20000

A011|Jody Mills|SIEM Monitoring|INC007|Phishing|3|2016-08-14|4|12800

A003|Sam Smith|Digital Forensics|INC008|Data Breach|5|2012-11-19|42|210000

A023|Sarah Butane|Malware Analysis|INC009|Trojan Infection|4|2021-02-10|9|55000

A002|Castiel Novak|SIEM Monitoring|INC010|Brute Force Attack|3|2013-06-21|6|24000

A008|Bobby Singer|Network Security|INC011|DDoS|4|2017-03-11|10|65000

A001|Dean Smith|Network Security|INC012|Port Scanning Attack|2|2014-01-09|2|5000

A017|Nora Bennett|Malware Analysis|INC013|Worm Attack|4|2018-07-22|11|35000

A010|Kevin Tran|Digital Forensics|INC014|Insider Threat|4|2019-04-15|28|52000

A007|Eileen Leahy|SIEM Monitoring|INC015|Suspicious Login|2|2020-09-30|1|6000

A013|Garth Fitzgerald II|Network Security|INC016|Firewall Breach|5|2015-12-01|15|120000

A005|Claire Smith Novak|Digital Forensics|INC017|Unauthorized Access|3|2022-02-17|6|30000

A019|Ethan Cole|SIEM Monitoring|INC018|Account Compromise|4|2018-08-12|9|45000

A016|Alex Romero|Network Security|INC019|Man-in-the-Middle|3|2016-05-20|5|32000

A012|Donna Hanscum|Digital Forensics|INC020|Data Breach|5|2021-10-05|48|220000

A009|Charlie Bradbury|Malware Analysis|INC021|Ransomware|5|2017-03-29|26|250000

A020|Benny Lafitte|Network Security|INC022|DDoS|4|2019-01-18|11|60000

A004|Jack Kline|Network Security|INC023|Firewall Breach|5|2013-09-27|14|110000

A018|Maya Chen|Digital Forensics|INC024|Evidence Tampering|5|2023-05-11|36|130000

A024|Thomas Mayo|SIEM Monitoring|INC025|Phishing|3|2011-04-07|3|10000

A006|Crowley Macleod|Malware Analysis|INC026|Spyware|3|2021-06-19|5|22000

A015|Rufus Turner|SIEM Monitoring|INC027|Brute Force Attack|3|2020-03-12|6|27000

A003|Sam Smith|Digital Forensics|INC028|Unauthorized Access|3|2018-07-01|7|40000

A011|Jody Mills|SIEM Monitoring|INC029|Suspicious Login|2|2015-02-14|2|5000

A017|Nora Bennett|Malware Analysis|INC030|Trojan Infection|4|2022-09-22|10|50000

A008|Bobby Singer|Network Security|INC031|Man-in-the-Middle|3|2014-12-11|5|32000

A010|Kevin Tran|Digital Forensics|INC032|Data Breach|5|2020-10-21|40|155000

A014|Rowena MacLeod|Malware Analysis|INC033|Ransomware|5|2022-06-05|25|95000

A001|Dean Smith|Network Security|INC034|Firewall Breach|5|2016-02-25|12|138000

A021|Kevin Moskowitz|Malware Analysis|INC035|Worm Attack|4|2021-07-05|10|68000

A004|Jack Kline|Network Security|INC036|DDoS|4|2018-11-15|9|46000

A005|Claire Smith Novak|Digital Forensics|INC037|Evidence Tampering|5|2017-08-21|38|125000

A012|Donna Hanscum|Digital Forensics|INC038|Insider Threat|4|2020-04-15|28|62000

A020|Benny Lafitte|Network Security|INC039|Port Scanning Attack|2|2020-08-14|2|5000

A016|Alex Romero|Network Security|INC040|DDoS|4|2015-07-23|10|64000

A007|Eileen Leahy|SIEM Monitoring|INC041|Phishing|3|2021-06-25|3|13000

A021|Kevin Moskowitz|Malware Analysis|INC042|Spyware|3|2020-11-14|5|21000

A002|Castiel Novak|SIEM Monitoring|INC043|Suspicious Login|2|2011-07-18|1|4000

A015|Rufus Turner|SIEM Monitoring|INC044|Brute Force Attack|3|2022-12-06|6|28000

A003|Sam Smith|Digital Forensics|INC045|Evidence Tampering|5|2023-08-12|36|123000

A013|Garth Fitzgerald II|Network Security|INC046|Port Scanning Attack|2|2012-08-21|2|3900

A014|Rowena MacLeod|Malware Analysis|INC047|Spyware|3|2022-06-01|5|19000

A011|Jody Mills|SIEM Monitoring|INC048|Account Compromise|4|2015-06-29|10|58000

A018|Maya Chen|Digital Forensics|INC049|Evidence Tampering|5|2024-03-30|42|120000

A001|Dean Smith|Network Security|INC050|Man-in-the-Middle|3|2011-05-16|6|15000

A023|Sarah Butane|Malware Analysis|INC051|Ransomware|5|2022-03-13|30|310000

A008|Bobby Singer|Network Security|INC052|DDoS|4|2019-04-13|12|66000

A010|Kevin Tran|Digital Forensics|INC053|Insider Threat|4|2019-07-11|28|52000

A017|Nora Bennett|Malware Analysis|INC054|Ransomware|5|2023-01-12|25|150000

A005|Claire Smith Novak|Digital Forensics|INC055|Unauthorized Access|3|2016-03-08|5|32000

A007|Eileen Leahy|SIEM Monitoring|INC056|Suspicious Login|2|2020-03-17|2|6000

A019|Ethan Cole|SIEM Monitoring|INC057|Brute Force Attack|3|2022-08-15|7|28000

A016|Alex Romero|Network Security|INC058|Firewall Breach|5|2023-06-11|18|78000

A012|Donna Hanscum|Digital Forensics|INC059|Data Breach|5|2025-02-24|50|230000

A009|Charlie Bradbury|Malware Analysis|INC060|Trojan Infection|4|2024-07-22|9|57000

A002|Castiel Novak|SIEM Monitoring|INC061|Phishing|3|2017-10-15|4|12000

A014|Rowena MacLeod|Malware Analysis|INC062|Ransomware|5|2021-06-20|28|98000

A013|Garth Fitzgerald II|Network Security|INC063|Firewall Breach|5|2017-03-21|16|122000

A015|Rufus Turner|SIEM Monitoring|INC064|Suspicious Login|2|2013-12-20|1|3500

A001|Dean Smith|Network Security|INC065|DDoS|4|2020-03-11|9|59000

A020|Benny Lafitte|Network Security|INC066|Port Scanning Attack|2|2022-01-11|2|5700

A024|Thomas Mayo|SIEM Monitoring|INC067|Phishing|3|2018-09-17|6|19000

A003|Sam Smith|Digital Forensics|INC068|Insider Threat|4|2024-02-06|30|59000

A004|Jack Kline|Network Security|INC069|DDoS|4|2018-11-15|9|46000

A006|Crowley Macleod|Malware Analysis|INC070|Spyware|3|2019-02-10|5|20000

A008|Bobby Singer|Network Security|INC071|Man-in-the-Middle|3|2013-05-09|5|14700

A010|Kevin Tran|Digital Forensics|INC072|Data Breach|5|2025-04-12|35|154000

A017|Nora Bennett|Malware Analysis|INC073|Worm Attack|4|2017-08-19|11|35000

A012|Donna Hanscum|Digital Forensics|INC074|Insider Threat|4|2020-04-15|28|62000

A007|Eileen Leahy|SIEM Monitoring|INC075|Brute Force Attack|3|2019-12-04|6|29000

A005|Claire Smith Novak|Digital Forensics|INC076|Evidence Tampering|5|2017-08-21|38|125000

A019|Ethan Cole|SIEM Monitoring|INC077|Phishing|3|2018-03-22|3|19000

A016|Alex Romero|Network Security|INC078|DDoS|4|2015-07-23|10|63000

A009|Charlie Bradbury|Malware Analysis|INC079|Ransomware|5|2015-03-22|20|220000

A014|Rowena MacLeod|Malware Analysis|INC080|Spyware|3|2022-06-01|5|19000

A002|Castiel Novak|SIEM Monitoring|INC081|Brute Force Attack|3|2012-05-16|7|25000

A003|Sam Smith|Digital Forensics|INC082|Evidence Tampering|5|2023-08-12|36|123000

A001|Dean Smith|Network Security|INC083|Firewall Breach|5|2016-08-11|18|145000

A018|Maya Chen|Digital Forensics|INC084|Evidence Tampering|5|2024-03-30|42|120000

A022|Alexander Jeremy|Malware Analysis|INC085|Ransomware|5|2022-03-13|30|310000

A013|Garth Fitzgerald II|Network Security|INC086|Port Scanning Attack|2|2012-08-21|2|3900

A020|Benny Lafitte|Network Security|INC087|DDoS|4|2018-09-02|11|67000

A010|Kevin Tran|Digital Forensics|INC088|Insider Threat|4|2019-07-11|28|51000

A011|Jody Mills|SIEM Monitoring|INC089|Account Compromise|4|2015-06-29|10|58000

A008|Bobby Singer|Network Security|INC090|DDoS|4|2019-04-13|12|66000

A012|Donna Hanscum|Digital Forensics|INC091|Data Breach|5|2025-02-24|50|230000

A007|Eileen Leahy|SIEM Monitoring|INC092|Suspicious Login|2|2020-03-17|2|6000

A024|Thomas Mayo|SIEM Monitoring|INC093|Brute Force Attack|3|2022-12-06|6|28000

A014|Rowena MacLeod|Malware Analysis|INC094|Ransomware|5|2023-01-12|25|150000

A009|Charlie Bradbury|Malware Analysis|INC095|Trojan Infection|4|2024-07-22|9|57000

A016|Alex Romero|Network Security|INC096|Firewall Breach|5|2023-06-11|18|78000

A017|Nora Bennett|Malware Analysis|INC097|Spyware|3|2018-05-30|6|20000

A003|Sam Smith|Digital Forensics|INC098|Unauthorized Access|3|2018-07-01|7|40000

A005|Claire Smith Novak|Digital Forensics|INC099|Evidence Tampering|5|2017-08-21|38|125000

A001|Dean Smith|Network Security|INC100|Firewall Breach|5|2016-08-11|18|145000

Java Code

The system is implemented using Java programming language. It consists of several classes and data structures such as LinkedList, Queue, and Stack.

AnalystInfo Class

```
import java.util.*;

public class AnalystInfo{

    private String analystId;

    private String analystName;

    private String expertiseArea;

    private List<IncidentInfo> incidents;


    public AnalystInfo(String analystId,String analystName,String
expertiseArea){

        this.analystId = analystId;

        this.analystName = analystName;

        this.expertiseArea = expertiseArea;

        this.incidents = new LinkedList<>();
```

```
    }

    public void addIncident(IncidentInfo incident) {
        incidents.add(incident);
    }

    public String getAnalystId(){
        return analystId;
    }

    public String getAnalystName(){ //for queue display
        return analystName;
    }

    public String getExpertiseArea(){ //for queue display
        return expertiseArea;
    }

    public List<IncidentInfo> getIncidents(){
        return incidents;
    }
    //requirement for queue display
    public double getTotalImpactCost(){
        double total = 0;
        for(IncidentInfo i: incidents){
            total += i.getImpactCost();
        }
        return total;
    }
}
```

IncidentInfo class:

```
public class IncidentInfo
```

```
{
    private String incidentId;
    private String incidentType;
    private int severityLevel;
    private String reportDate;
    private String estimatedResolutionTime;
    private double impactCost;

    public IncidentInfo(String incidentId,String incidentType,int
severityLevel,String reportDate,String
estimatedResolutionTime,double impactCost){
        this.incidentId = incidentId;
        this.incidentType = incidentType;
        this.severityLevel = severityLevel;
        this.reportDate = reportDate;
        this.estimatedResolutionTime = estimatedResolutionTime;
        this.impactCost = impactCost;
    }
    //for queue display
    public String getIncidentId() {
        return incidentId;
    }
    public String getIncidentType() {
        return incidentType;
    }
    public int getSeverityLevel() {
        return severityLevel;
    }
    public String getReportDate() {
        return reportDate;
    }
    public String getERT() { //ERT = estimatedResolutionTime
        return estimatedResolutionTime;
```

```
    }
    public double getImpactCost() {
        return impactCost;
    }
}
```

IncidentRouting class:

```
import java.util.*;


public class IncidentRouting {


    // Using Java's built-in Queue interface with LinkedList
    private Queue<AnalystInfo> internalQueue = new LinkedList<>();
// Queue 1
    private Queue<AnalystInfo> externalQueue = new LinkedList<>();
// Queue 2
    private Queue<AnalystInfo> criticalQueue = new LinkedList<>();
// Queue 3


    private List<AnalystInfo> analystList;


    public IncidentRouting(List<AnalystInfo> analystList) {
        this.analystList = analystList;
    }


    // Route analysts based on incident count
    public void routeAnalysts() {
        boolean assignToInternal = true;


        for (AnalystInfo analyst : analystList) {
            if (analyst.getIncidents().size() > 3) {
                criticalQueue.add(analyst);
                System.out.println(analyst.getAnalystName() + " ->
Critical Queue");
            } else {
```

```
                if (assignToInternal) {
                    internalQueue.add(analyst);
                    System.out.println(analyst.getAnalystName() + "
-> Internal Queue");
                } else {
                    externalQueue.add(analyst);
                    System.out.println(analyst.getAnalystName() + "
-> External Queue");
                }
                assignToInternal = !assignToInternal;
            }
        }
    }

    // Helper method to display queue details
    public  List<String> getQueueDetails(Queue<AnalystInfo> queue) {
        List<String> details = new ArrayList<>();

        if (queue.isEmpty()) {
            details.add("Queue is empty");
            return details;
        }

        for (AnalystInfo analyst : queue) {

            StringBuilder text = new StringBuilder();
            text.append("\nAnalyst ID "
).append(analyst.getAnalystId()).append("\n");
            text.append("  Name:
").append(analyst.getAnalystName()).append("\n");
            text.append("  Expertise:
").append(analyst.getExpertiseArea()).append("\n");
            text.append("  Assigned Incidents:\n ");
```

```
            for(IncidentInfo incident : analyst.getIncidents()){
                text.append("-").append(incident.getIncidentId())
                .append("
(").append(incident.getIncidentType()).append(")\n");
           text.append("-Impact Cost:
RM").append(String.format("%.2f",incident.getImpactCost())).append("
\n\n");
            }

            double totalCost = analyst.getTotalImpactCost();

            text.append("TOTAL IMPACT COST: RM
").append(String.format("%.2f",totalCost)).append("\n");

            text.append("-----------------------------");
            details.add(text.toString());
        }

        return details;
    }

    // Getters for Phase 3
    public Queue<AnalystInfo> getInternalQueue() {
        return internalQueue;
    }

    public Queue<AnalystInfo> getExternalQueue() {
        return externalQueue;
    }

    public Queue<AnalystInfo> getCriticalQueue() {
        return criticalQueue;
    }
}
```

IncidentResolution class:

```
import java.util.*;
public class IncidentResolution
{
    private Stack<AnalystInfo> resolvedStack;

    public IncidentResolution()
    {
        resolvedStack = new Stack<>();
    }

    public void processIncidents(Queue<AnalystInfo> internalQueue,
                                 Queue<AnalystInfo> externalQueue,
                                 Queue<AnalystInfo> criticalQueue)
    {
       int batchNum = 1;

        // Repeat until all queues are empty
        while (!internalQueue.isEmpty() || !externalQueue.isEmpty()
|| !criticalQueue.isEmpty()) {

            System.out.println("\n========== BATCH " + batchNum + "
==========");

            serveFromQueue(internalQueue, "Internal Queue", 5);
            serveFromQueue(externalQueue, "External Queue", 5);
            serveFromQueue(criticalQueue, "Critical Queue", 5);

            batchNum++;
        }
```

```
System.out.println("\n==============================================
==================");

        System.out.println("ALL QUEUES ARE EMPTY");

System.out.println("================================================
================");


        displayResolvedStack();

    }


    /**

     * Serve analysts from a queue (FIFO)

     */

    private void serveFromQueue(Queue<AnalystInfo> queue, String
queueName, int batchSize)

    {

        System.out.println("\n--- " + queueName + " ---");

        System.out.println("Analysts waiting: " + queue.size());


        if (queue.isEmpty()) {

            System.out.println("No analysts in this queue.");

            return;

        }


        int served = 0;


        while (!queue.isEmpty() && served < batchSize) {


            AnalystInfo analyst = queue.poll(); // FIFO


            System.out.println("\nServing: " +
analyst.getAnalystName() + " (" + analyst.getAnalystId() + ")");

            System.out.println("  Expertise: " +
analyst.getExpertiseArea());
```

```
            System.out.println("  Number of incidents: " +
analyst.getIncidents().size());


            double totalImpact = completeIncidents(analyst);


            resolvedStack.push(analyst); // LIFO

            System.out.println("  >> PUSHED to resolvedStack (Stack
size: " + resolvedStack.size() + ")");


            served++;

        }


        System.out.println("\nServed " + served + " analyst(s) from
" + queueName);


        if (queue.isEmpty()) {

            System.out.println(queueName + " is now EMPTY.");

        }

    }


    /**

     * Complete all incidents for an analyst

     */

    private double completeIncidents(AnalystInfo analyst)

    {

        List<IncidentInfo> incidents = analyst.getIncidents();

        double totalImpact = 0;


        System.out.println("  Incidents handled:");


        for (int i = 0; i < incidents.size(); i++) {

            IncidentInfo inc = incidents.get(i);

            totalImpact += inc.getImpactCost();
```

```
            System.out.printf("    %d. %s - %s (RM %,.2f)%n",
                (i + 1),
                inc.getIncidentId(),
                inc.getIncidentType(),
                inc.getImpactCost());
        }

        System.out.printf("  Total impact cost resolved: RM
%,.2f%n", totalImpact);

        return totalImpact;
    }

    /**
     * Display resolved stack (LIFO)
     */
    private void displayResolvedStack()
    {
System.out.println("\n==============================================
==================");
        System.out.println("RESOLVED STACK (LIFO Order - Most Recent
First)");
System.out.println("==============================================
================");

        if (resolvedStack.isEmpty()) {
            System.out.println("No analysts in resolvedStack.");
            return;
        }

        Stack<AnalystInfo> tempStack = new Stack<>();
        int position = 1;
        double grandTotal = 0;
```

```
        System.out.println();

        while (!resolvedStack.isEmpty()) {
            AnalystInfo analyst = resolvedStack.pop();
            tempStack.push(analyst);

            System.out.println("[" + position + "] " +
analyst.getAnalystName() + " (" + analyst.getAnalystId() + ")");
            System.out.println("    Expertise Area: " +
analyst.getExpertiseArea());
            System.out.println("    Incidents Handled: " +
analyst.getIncidents().size());

            // show incident details
            System.out.println("    Incident Details:");
            for (IncidentInfo inc : analyst.getIncidents()) {
                System.out.printf("      - %s (%s) RM %,.2f%n",
                    inc.getIncidentId(),
                    inc.getIncidentType(),
                    inc.getImpactCost());
            }
            double totalCost= analyst.getTotalImpactCost();

            System.out.printf("    Total Impact Cost Resolved: RM
%,.2f%n",totalCost);
            System.out.println();

            grandTotal += totalCost;
            position++;
        }

        // Restore stack
        while (!tempStack.isEmpty()) {
```

```
            resolvedStack.push(tempStack.pop());
        }



System.out.println("================================================
================");
        System.out.printf("GRAND TOTAL IMPACT COST: RM %,.2f%n",
grandTotal);

System.out.println("================================================
================");
    }


    public Stack<AnalystInfo> getResolvedStack()
    {
        return resolvedStack;
    }
}
```

CaseReader class:

```
import java.io.*;
import java.util.*;


public class CaseReader
{
    public static LinkedList<AnalystInfo> readFromFile(String
filename){
        //store all analysts in LinkedList
        LinkedList<AnalystInfo>analystList = new LinkedList<>();


        try(BufferedReader reader = new BufferedReader(new
FileReader(filename))){
            String line;


            //read line by line
```

```
            while((line=reader.readLine()) != null){
                if (line.trim().isEmpty()) {
                    continue;
                }
                //delimiter is '|'
                String[]parts = line.split("\\|");

                if (parts.length != 9){//information must have 9
parts
                    System.out.println("Invalid Data"+ line);
                    continue;
                }
                //analyst Information
                String analystId = parts[0];
                String analystName = parts[1];
                String expertiseArea = parts[2];

                //incidents Information
                String incidentId = parts[3];
                String incidentType = parts[4];
                int severityLevel = Integer.parseInt(parts[5]);
                String reportDate = parts[6];
                String estimatedResolutionTime = parts[7];
                double impactCost = Double.parseDouble(parts[8]);

                //Incident object
                IncidentInfo incident = new IncidentInfo(

incidentId,incidentType,severityLevel,reportDate,estimatedResolution
Time,impactCost
                    );

                //check if analyst already exists
                AnalystInfo analyst = null;
```

```
                for(AnalystInfo a: analystList){
                    if(a.getAnalystId().equals(analystId)){
                        analyst=a;
                        break;
                    }
                }
                //if analyst doesn't exist yet
                if(analyst == null){
                    //create new
                    analyst = new AnalystInfo(analystId,
analystName, expertiseArea);
                    analystList.add(analyst);
                }
                //add incident to that analyst
                analyst.addIncident(incident);


            }
        }catch(FileNotFoundException e){
            System.out.println("could not locate file");
        }
        catch(IOException e){
            System.out.println("Something went wrong ");
            e.printStackTrace();
        }
        return analystList;
    }
}
```

MainGUI(main class):

```
import javax.swing.*;
import java.awt.*;
import java.util.*;
```

```
public class MainGUI extends JFrame {

    private IncidentRouting routing;

    private JPanel internalPanel, externalPanel, criticalPanel;

    private Stack<AnalystInfo> resolvedStack = new Stack<>();

    public MainGUI() {

        setTitle("SOC Incident Management");
        setSize(1000, 700);
        setLayout(new BorderLayout());

        // ===== Panels =====
        internalPanel = createPanel("Internal Queue");
        externalPanel = createPanel("External Queue");
        criticalPanel = createPanel("Critical Queue");

        JPanel centerPanel = new JPanel(new GridLayout(1, 3));
        centerPanel.add(new JScrollPane(internalPanel));
        centerPanel.add(new JScrollPane(externalPanel));
        centerPanel.add(new JScrollPane(criticalPanel));

        add(centerPanel, BorderLayout.CENTER);

        // ===== Buttons =====
        JButton loadBtn = new JButton("Load File");
        JButton showBtn = new JButton("Show Queue");
        JButton nextBtn = new JButton("Next Queue");
        JButton stackBtn = new JButton("Show Completed");
```

```
        JPanel btnPanel = new JPanel();
        btnPanel.add(loadBtn);
        btnPanel.add(showBtn);
        btnPanel.add(nextBtn);
        btnPanel.add(stackBtn);

        add(btnPanel, BorderLayout.SOUTH);

        // ===== Actions =====

        loadBtn.addActionListener(e -> loadData());

        showBtn.addActionListener(e -> {
                    displayQueue(routing.getInternalQueue(),
internalPanel);
                    displayQueue(routing.getExternalQueue(),
externalPanel);
                    displayQueue(routing.getCriticalQueue(),
criticalPanel);
            });

        nextBtn.addActionListener(e -> processNextBatch());

        stackBtn.addActionListener(e -> showResolvedStack());

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    // ===== Load Data =====
    private void loadData() {

        LinkedList<AnalystInfo> analystList =
            CaseReader.readFromFile("cyber_incidents.txt");
```

```
        routing = new IncidentRouting(analystList);

        routing.routeAnalysts();

        JOptionPane.showMessageDialog(this, "File Loaded & Queues
Created!");
    }

    // ===== Display Queue (max 5 only) =====
    private void displayQueue(Queue<AnalystInfo> queue, JPanel
panel) {

        panel.removeAll();

        java.util.List<String> details
=routing.getQueueDetails(queue);

        int count = 0;

        for (String text : details) {

            if (count == 5) break;

            JTextArea area = new JTextArea(text);
            area.setEditable(false);

            panel.add(area);
            count++;
        }

        panel.revalidate();
        panel.repaint();
    }
```

```
    // ===== Process Next Batch =====
    private void processNextBatch() {

        processQueue(routing.getInternalQueue());
        processQueue(routing.getExternalQueue());
        processQueue(routing.getCriticalQueue());

        displayQueue(routing.getInternalQueue(), internalPanel);
        displayQueue(routing.getExternalQueue(), externalPanel);
        displayQueue(routing.getCriticalQueue(), criticalPanel);
    }

    // ===== Process 5 analysts =====
    private void processQueue(Queue<AnalystInfo> queue) {

        int count = 0;

        while (!queue.isEmpty() && count < 5) {

            AnalystInfo analyst = queue.poll();

            System.out.println("Processing: " +
analyst.getAnalystName());
            // push into stack (completed)
            resolvedStack.push(analyst);

            count++;
        }
    }
    //display stack
    private void showResolvedStack() {
```

```
        if(resolvedStack.isEmpty()){
            JOptionPane.showMessageDialog(this," No completed
analysts yet.");
            return;
        }

        StringBuilder text = new StringBuilder();

        Stack<AnalystInfo> temp = new Stack<>();

        text.append("\n=== COMPLETED ANALYSTS (STACK) ===\n");

        while (!resolvedStack.isEmpty()) {

            AnalystInfo analyst = resolvedStack.pop();
            temp.push(analyst);

            text.append(" Name:
").append(analyst.getAnalystName()).append("\n");
            text.append(" Expertise:
").append(analyst.getExpertiseArea()).append("\n");

            double total = 0;

            for (IncidentInfo i : analyst.getIncidents()) {
                text.append(" -
").append(i.getIncidentId()).append("
(").append(i.getIncidentType()).append(")")
                .append(" Severity
Lv.(").append(i.getSeverityLevel()).append(")\n")
                .append(" Report Date :
").append(i.getReportDate()).append("\n")
                .append(" Est. Resolution Time(Hour):
").append(i.getERT()).append("\n");
                total += i.getImpactCost();
```

```
            }

            text.append(" Total Cost: RM
").append(String.format("%.2f",total)).append("\n\n");
            text.append("--------------------------\n");
        }
        //restore stack
        while(!temp.isEmpty()){
            resolvedStack.push(temp.pop());
        }
        JTextArea area = new JTextArea(text.toString());
        area.setEditable(false);
        area.setLineWrap(true);
        area.setWrapStyleWord(true);

        JScrollPane scroll = new JScrollPane(area);
        scroll.setPreferredSize(new Dimension(500, 400));

        JOptionPane.showMessageDialog(this,scroll,"Resolved
Stack",JOptionPane.INFORMATION_MESSAGE);
    }
    // ===== Panel helper =====
    private JPanel createPanel(String title) {
        JPanel panel = new JPanel();
        panel.setBorder(BorderFactory.createTitledBorder(title));
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        return panel;
    }

    public static void main(String[] args) {
        new MainGUI();
    }
}
```

SAMPLE INPUT / OUTPUT

The system reads data from a text file containing analyst and incident information. Each line represents one incident.

Input:

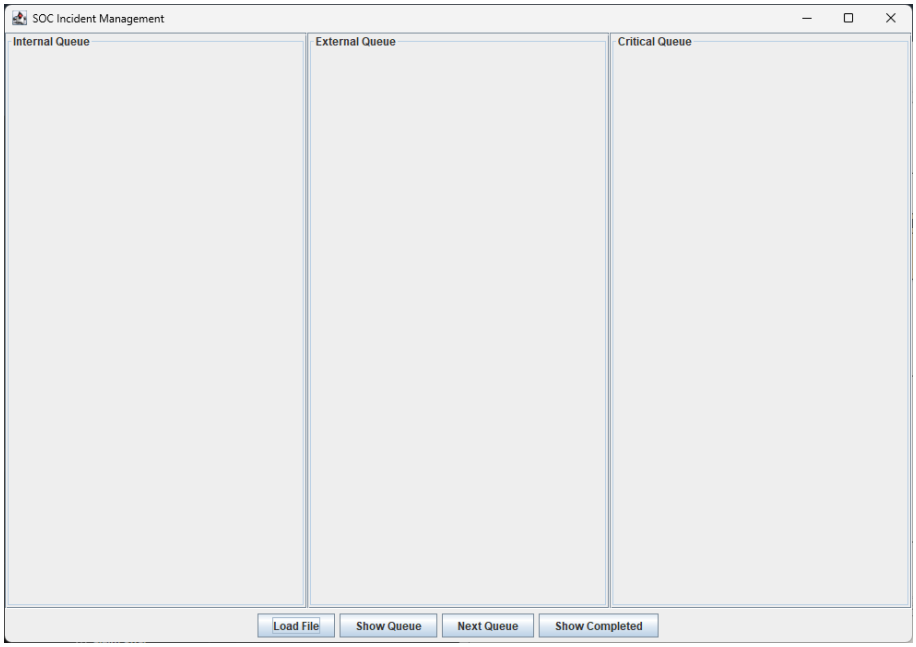
cyber_incidents.txt - GroupProject2026(2)

Class Edit Tools Options

CaseReader | cyber_incidents.txt

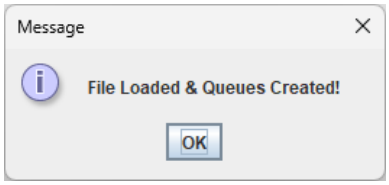
Compile Undo Cut Copy Paste Find... Close

```
A001|Dean Smith|Network Security|INC001|Port Scanning Attack|2|2009-03-26|3|5000
A002|Castiel Novak|SIEM Monitoring|INC002|Brute Force Attack|3|2010-09-06|7|25000
A003|Sam Smith|Digital Forensics|INC003|Insider Threat|4|2009-03-26|5|100000
A004|Jack Kline|Network Security|INC004|DDoS|4|2017-05-18|5|40000
A005|Claire Smith Novak|Digital Forensics|INC005|Unauthorized Access|3|2023-04-09|5|30000
A006|Crowley Macleod|Malware Analysis|INC006|Spyware|3|2020-05-30|6|20000
A011|Jody Mills|SIEM Monitoring|INC007|Phishing|3|2016-08-14|4|12800
A003|Sam Smith|Digital Forensics|INC008|Data Breach|5|2012-11-19|42|210000
A023|Sarah Butane|Malware Analysis|INC009|Trojan Infection|4|2021-02-10|9|55000
A002|Castiel Novak|SIEM Monitoring|INC010|Brute Force Attack|3|2013-06-21|6|24000
A008|Bobby Singer|Network Security|INC011|DDoS|4|2017-03-11|10|65000
A001|Dean Smith|Network Security|INC012|Port Scanning Attack|2|2014-01-09|2|5000
A017|Nora Bennett|Malware Analysis|INC013|Worm Attack|4|2018-07-22|11|35000
A010|Kevin Tran|Digital Forensics|INC014|Insider Threat|4|2019-04-15|28|52000
A007|Eileen Leahy|SIEM Monitoring|INC015|Suspicious Login|2|2020-09-30|1|6000
A013|Garth Fitzgerald II|Network Security|INC016|Firewall Breach|5|2015-12-01|15|120000
A005|Claire Smith Novak|Digital Forensics|INC017|Unauthorized Access|3|2022-02-17|6|30000
A019|Ethan Cole|SIEM Monitoring|INC018|Account Compromise|4|2018-08-12|9|45000
A016|Alex Romero|Network Security|INC019|Man-in-the-Middle|3|2016-05-20|5|32000
A012|Donna Hanscum|Digital Forensics|INC020|Data Breach|5|2021-10-05|48|220000
A009|Charlie Bradbury|Malware Analysis|INC021|Ransomware|5|2017-03-29|26|250000
A020|Benny Lafitte|Network Security|INC022|DDoS|4|2019-01-18|11|60000
A004|Jack Kline|Network Security|INC023|Firewall Breach|5|2013-09-27|14|110000
A018|Maya Chen|Digital Forensics|INC024|Evidence Tampering|5|2023-05-11|36|130000
A024|Thomas Mayo|SIEM Monitoring|INC025|Phishing|3|2011-04-07|3|10000
A006|Crowley Macleod|Malware Analysis|INC026|Spyware|3|2021-06-19|5|22000
A015|Rufus Turner|SIEM Monitoring|INC027|Brute Force Attack|3|2020-03-12|6|27000
A003|Sam Smith|Digital Forensics|INC028|Unauthorized Access|3|2018-07-01|7|40000
A011|Jody Mills|SIEM Monitoring|INC029|Suspicious Login|2|2015-02-14|2|5000
A017|Nora Bennett|Malware Analysis|INC030|Trojan Infection|4|2022-09-22|10|50000
A008|Bobby Singer|Network Security|INC031|Man-in-the-Middle|3|2014-12-11|5|32000
```

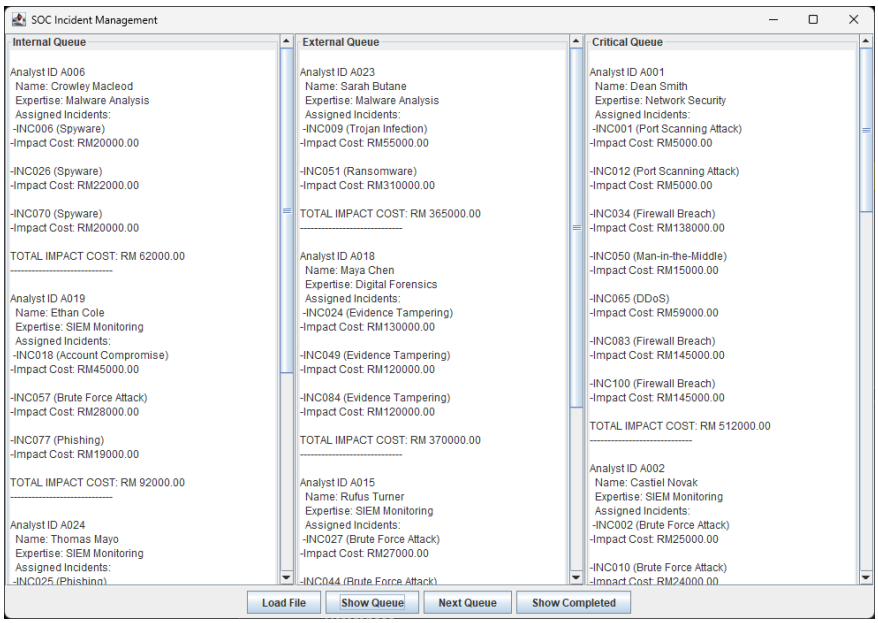


The second picture above is the look of the queue before load any files.

After click on Load File button, a message appeared that a queue has been created. This means the system read data from the cyber_incidents.txt file andcreateAnalyst amd incident objects, before storing them in a LinkedList.



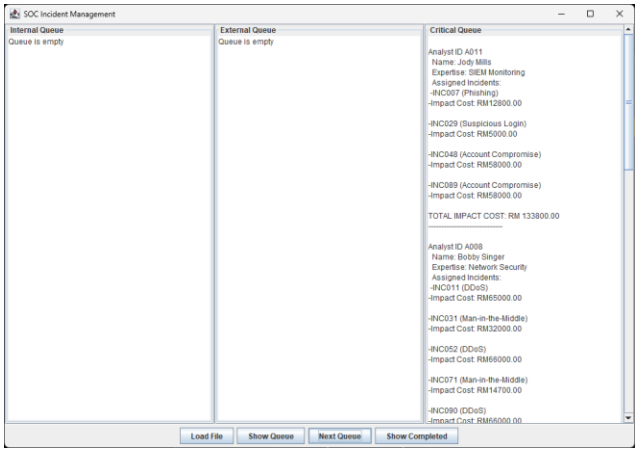
After clicking ‘Show Queue’ button, maximum 5 analyst from each 3 queues will be display according to each category. The system route analysts based on the number of incidents.

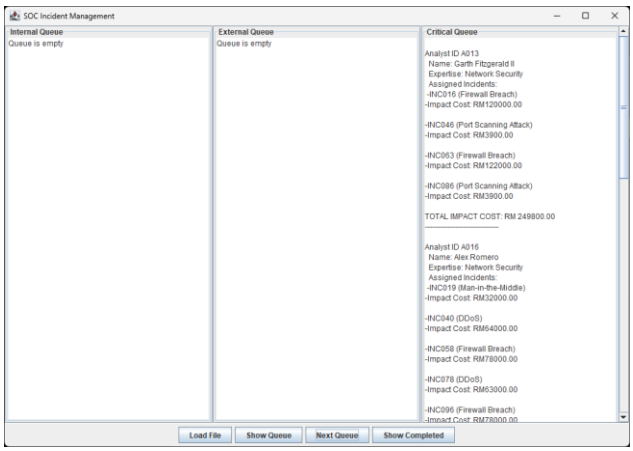


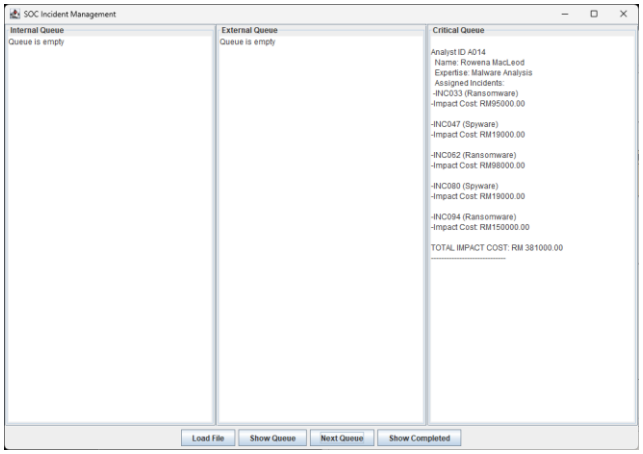
If an analyst has three or fewer incidents, they are placed between internal and external queues alternatively.

If an analyst has more than three incidents, they are placed into the Critical queue. Each panel shows the analyst’s name, expertise area, list of incidents, and the impact cost of a incident.

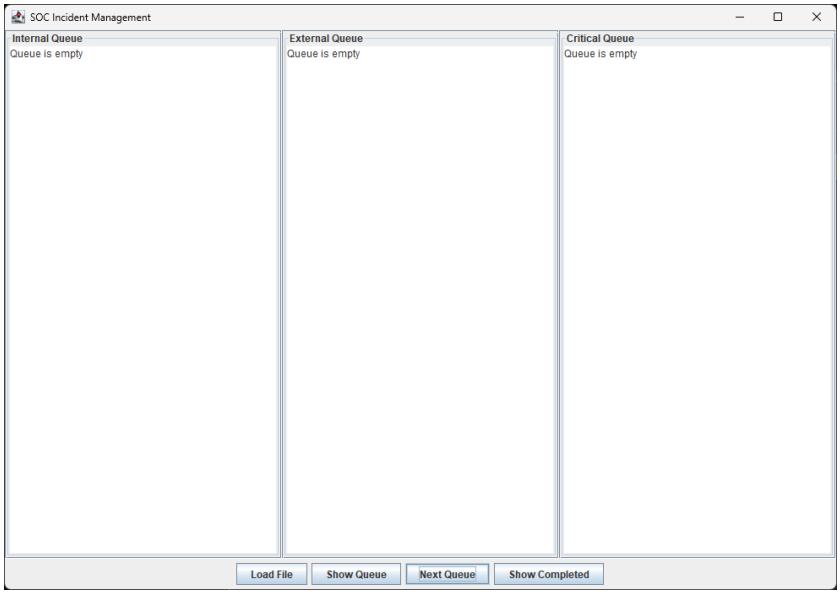
Clicking ‘Next Queue’ button will show the next queue.





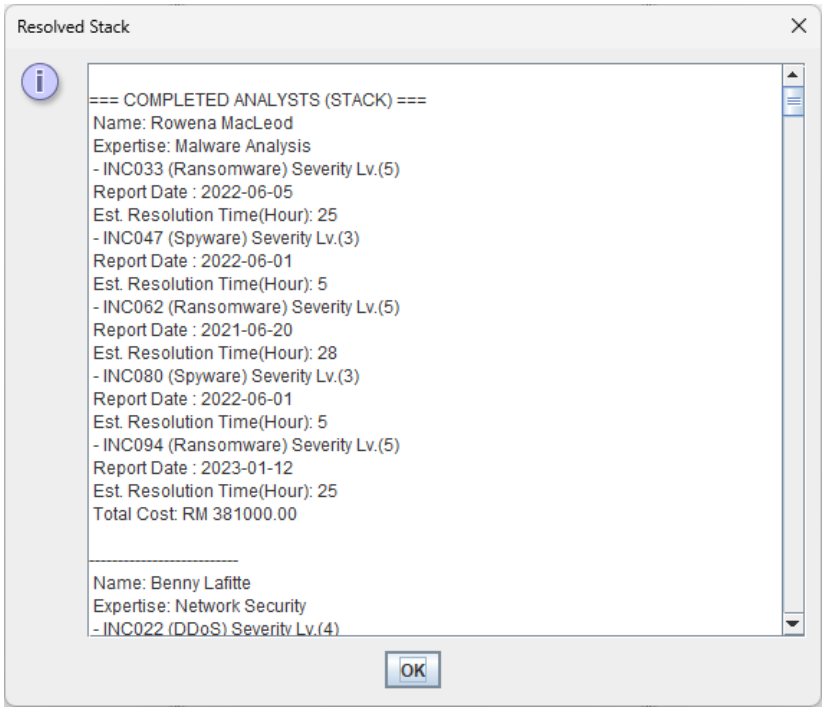


The system processes up to five analyst each queue. This follow the First In First Out, meaning the first analyst in the queue is processed first. This is done using queue.poll(). The same thing happen until all queue is completed.



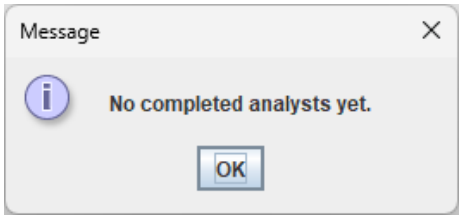
After processing, each analyst is pushed into a stack. This uses stack.push(), which follows LIFO,Last In, First Out. The most recently processed analyst will be displayed first later.

By clicking Show Completed, it will display all completed analysts.



The system pops analysts from the stack using stack.pop(). Each analyst's details and total impact cost are displayed in a scrollable window.

If 'Show Completed' click when no Analyst compete processing, a message will appear.



5. LESSON LEARNED FROM EACH MEMBER

<u>Adriana Khayrin Binti Mohd Sidek</u>

My name is Adriana Khayrin Binti Mohd Sidek, and I'm the leader for this group project.

I'm in charge of leading this group and assigned the task to each member in the group. As of individual tasks, I'm in charge of doing Data Modeling and created the GUI for the project.

During the making of this project, I learned how to apply different data structures such as LinkedList, Queue, and Stack in a real-world scenario. I understood how LinkedList is useful for storing dynamic data, while Queue and Stack help manage processing orders using FIFO and LIFO concepts.

I also improved my skills in Java programming, especially in handling files input, GUI development and organizing code into multiple classes. Additionally, I learned how to debug errors and ensure that different components of the system work together correctly.

One big issue I have during the making of the project is the ability to work together with other people on the coding process. I solved this issue by making sure I told the team member what

part of the coding they in charged with, and working the coding using Github to manage and merge our code differently. This improved collaboration.

In terms of team management, I learned the responsibility of being a team leader and how to guide my teammates and work efficiently.

Overall, despite all the challenges faced during development, this project provided valuable experience in both technical implementation and teamwork, and it enhanced my ability to manage a group project effectively.

Nur Aina Batrisya Binti Zulfikar

Hello my name is Nur Aina Batrisya and my contribution is on phase 3.

For Phase 3, I focused on using the Stack data structure to handle the incident resolution process. From this, I learned how the LIFO (Last-In-First-Out) concept works in a real application, where the most recently completed analyst is recorded first. I also understood how analysts are processed in batches from different queues before being pushed into the stack, which gave me a clearer picture of how different data structures can work together in one system.

Throughout this phase, I also developed my problem-solving and debugging skills, especially when ensuring that the stack correctly stored and displayed all the required information such as analyst details and total impact cost. It required careful checking to make sure everything was accurate. Overall, this experience helped me become more confident in handling more complex program logic and applying theoretical concepts into practical use.

Saidatul Nazirah Binti Razak

Hi, my name is Saidatul Nazirah and Im responsible for phase 2 on this project.

In Phase 2, I worked on using the Queue data structure to manage how analysts are assigned to different incident queues. From this, I learned how queues work in a practical way, especially the FIFO concept where data is processed in order. I also understood how to apply conditions to decide which queue an analyst should go into based on the number of incidents they handle. This helped me improve my logical thinking and problem-solving skills.

Besides that, I gained experience in handling multiple queues at the same time and making sure the data is organized correctly. I also had to test and debug my code to ensure everything worked as expected, especially when displaying the analyst details and total impact cost. Overall, this phase helped me better understand how queues can be used in real systems and improved my confidence in coding.

Qistina Batrisyia Binti Mohd Najib

My name is Qistina Batrisyia and my contribution is working on this project report.

For this project, I was responsible for preparing and organizing the report. Through this role, I learned that documentation is not just about writing, but also about clearly presenting the overall system and its process. I had to ensure that all sections such as the introduction, diagrams, explanations, and outputs were well-structured and easy to understand.

During the process, I improved my ability to explain technical concepts in a simpler and more organized way. I also learned how to follow the required format and maintain consistency throughout the report. This required careful checking to make sure all information matched the system developed by my group members.

Besides that, I developed better attention to detail and time management skills to complete the report within the given deadline. Overall, this experience helped me understand the importance of proper documentation and made me more confident in presenting project work in a clear and professional manner.

6. REFERENCES (APA FORMAT)

References

1. Oracle. (2024). Java documentation. https://docs.oracle.com/en/java/

2. Schildt, H. (2019). Java: The complete reference (11th ed.). McGraw-Hill Education.

3. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Java (6th ed.). Wiley.

4. GeeksforGeeks. (2024). Queue data structure. https://www.geeksforgeeks.org/queue-data-structure/

5. GeeksforGeeks. (2024). Stack data structure. https://www.geeksforgeeks.org/stack-data-structure/

6. GeeksforGeeks. (2024). Linked list in Java. https://www.geeksforgeeks.org/linked-list-in-java/

7. TutorialsPoint. (2024). Java data structures. https://www.tutorialspoint.com/java/java_data_structures.htm

VIDEO PRESENTATION LINK